iamneo

Advanced DataFrame Operations and Spark SQL

Advanced DataFrame Operations Overview

Objective:

The goal of this session is to understand and apply advanced DataFrame operations for complex data transformations, enhancing your ability to work with large datasets in Spark.

Topics Covered:

1. **Joins**:
   Learn how to combine DataFrames based on common keys.
2. **Unions**:
   Combine rows of two DataFrames with the same schema.
3. **Handling Missing Data**:
   Techniques to clean and handle null values.
4. **Spark SQL**:
   Introduction to SQL queries on DataFrames using Spark SQL.

This session builds on the basics by covering more complex operations and optimizing the performance of your Spark jobs.

DataFrame Joins

What are Joins?

A **Join** is the operation of combining rows from two or more DataFrames based on a related column. It's crucial when you need to link related data, such as combining employee records with their corresponding department information.

Types of Joins:

- **Inner Join**:
  Returns only rows with matching values in both DataFrames.
- **Left Outer Join**:
  Includes all rows from the left DataFrame and matching rows from the right. Non-matching rows from the right will have null values.
- **Right Outer Join**:
  Similar to Left Join, but includes all rows from the right DataFrame.
- **Full Outer Join**:
  Combines rows from both DataFrames. Rows without matches from either DataFrame will have null values in their respective columns.

DataFrame Joins

Example:

```
# Creating two DataFrames
df1 = spark.createDataFrame([(1, "Alice"), (2, "Bob")], ["id", "name"])
df2 = spark.createDataFrame([(1, "Sales"), (3, "HR")], ["id", "department"])

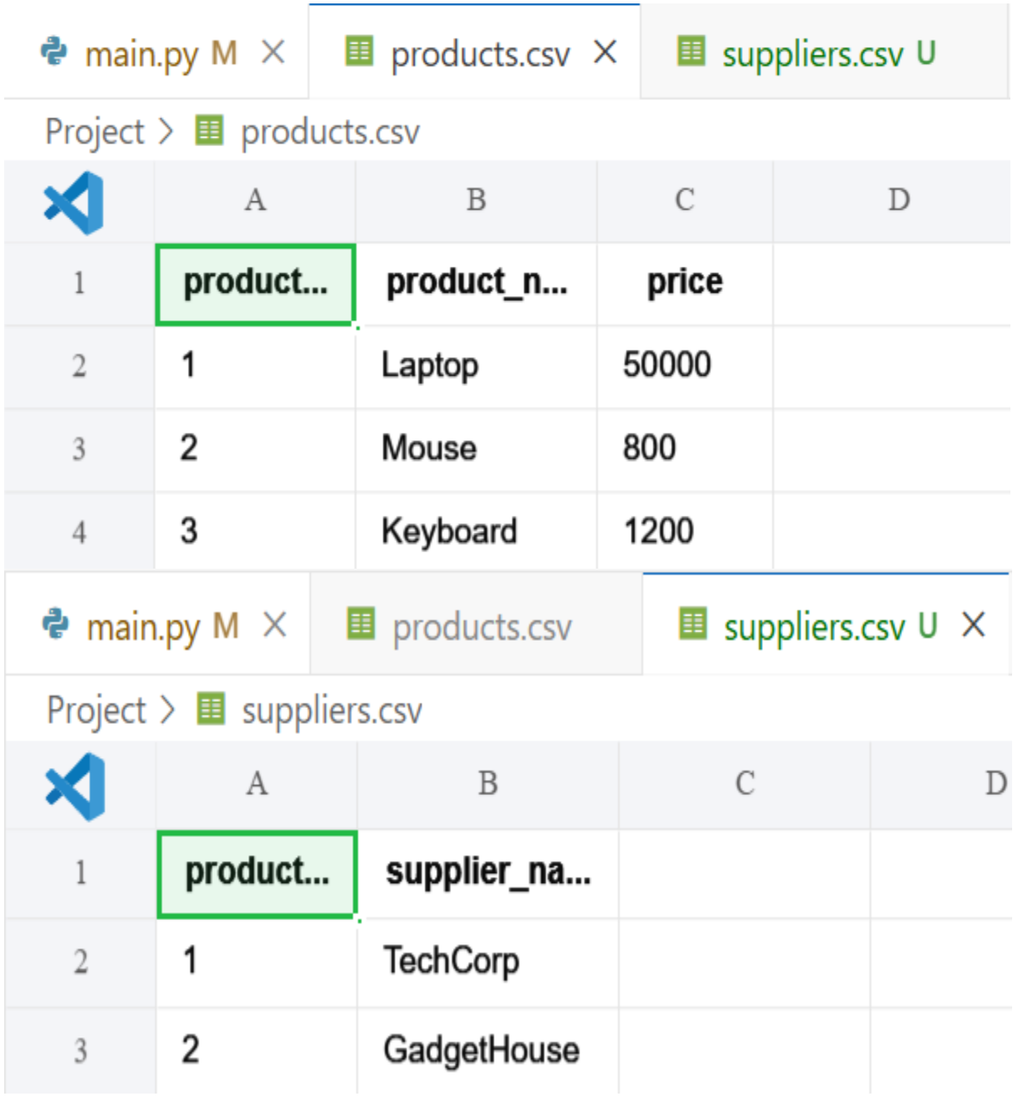
# Performing an inner join
joined_df = df1.join(df2, "id", "inner")
joined_df.show()
```

Output:

```
+--+----+----------+
| id| name|department|
+--+----+----------+
|  1|Alice|   Sales|
+--+----+----------+
```

Example: Inner Join – Products with Supplier Names

- Reads product and supplier data from two separate CSV files.
- Performs an inner join using the common key product_id.
- Only products with matching supplier entries are included in the output.



Example: Inner Join – Products with Supplier Names (Solution)

main.py M | products.csv | suppliers.csv U

Project > main.py > ...

```python
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("Product Supplier Join").getOrCreate()

# Load CSVs into DataFrames
products = spark.read.option("header", "true").csv("products.csv", inferSchema=True)
suppliers = spark.read.option("header", "true").csv("suppliers.csv", inferSchema=True)

# Perform inner join on product_id
joined_df = products.join(suppliers, "product_id", "inner")

# Show result
joined_df.show()
```

```
+----------+------------+-----+-------------+
|product_id|product_name|price|supplier_name|
+----------+------------+-----+-------------+
|         1|      Laptop|50000|     TechCorp|
|         2|       Mouse|  800|  GadgetHouse|
+----------+------------+-----+-------------+

coder@dcfacefbacddcf329981901fdbaacaacdabdabdtwo-0:~/project/workspace/Project$
```

Joins in Action - Use Cases

Practical Examples:

- **Employee Details with Department Info**:
  - Merging employee information with department names. Using a join helps link the employees with their corresponding departments.
- **Sales Data with Regional Information**:
  - Suppose we have a sales DataFrame and another DataFrame containing regional information. A join would allow us to correlate sales with the correct region.

Joins in Action - Use Cases

Hands-On Exercise:

- **Task:** Perform a left join on two DataFrames to merge customer data with order history.

```
# Create customer and orders DataFrames
df_customers = spark.createDataFrame([(1, "Alice"), (2, "Bob")], ["customer_id", "name"])
df_orders = spark.createDataFrame([(1, 100), (2, 200)], ["customer_id", "order_value"])

# Perform a left join
df_customers.join(df_orders, "customer_id", "left").show()
```

Union of DataFrames

Union Operation:
The union operation combines two DataFrames, provided they have the same schema (same column names and types). Unlike a join, union stacks DataFrames on top of each other.
Example:

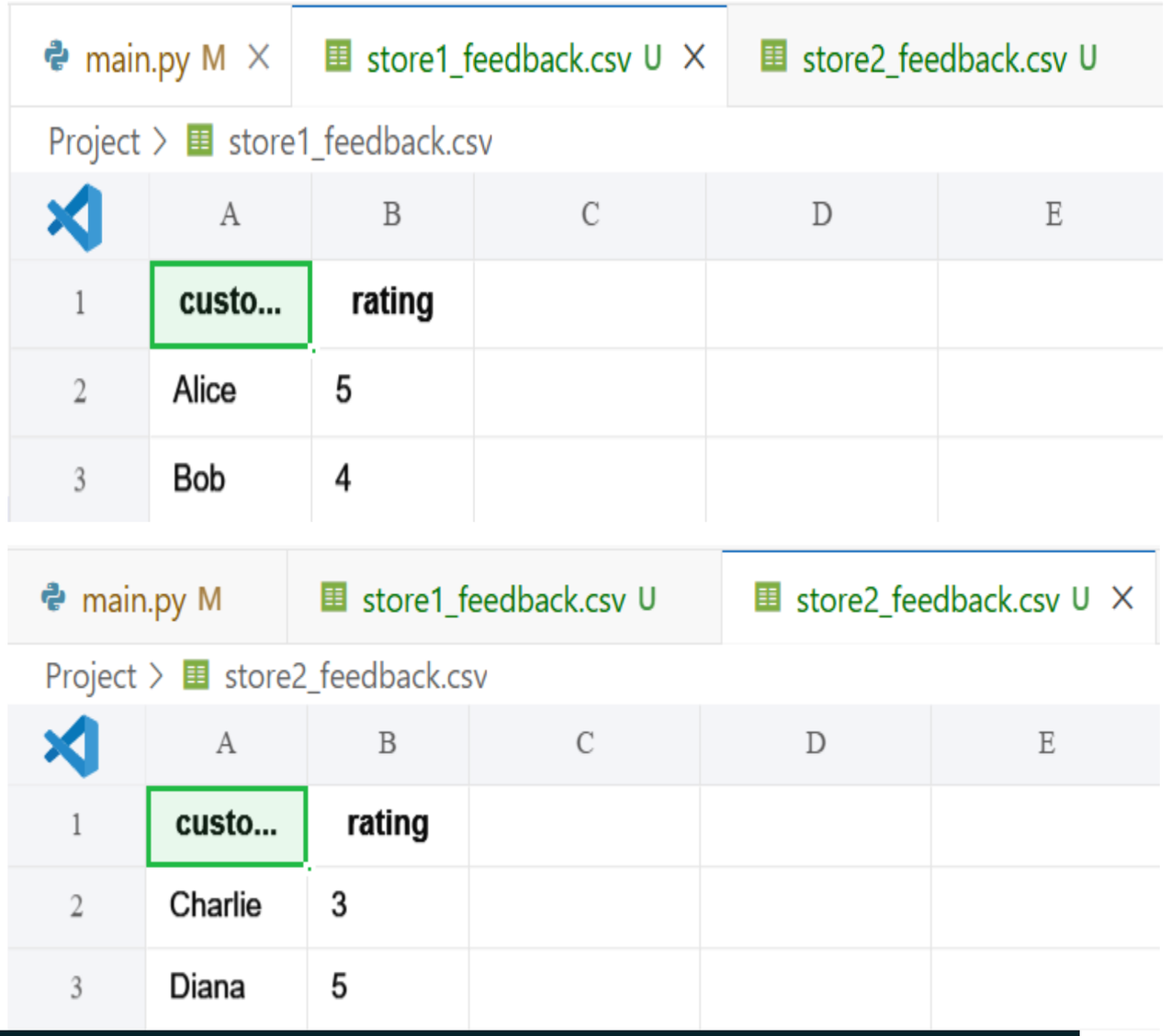
```
# Creating two DataFrames with the same schema
df1 = spark.createDataFrame([(1, "Alice")], ["id", "name"])
df2 = spark.createDataFrame([(2, "Bob")], ["id", "name"])
# Perform union
union_df = df1.union(df2)
union_df.show()
```

Output:

```
+---+-----+
| id| name|
+---+-----+
|  1|Alice|
|  2|  Bob|
+---+-----+
```

Example: Union – Merge Customer Feedback from Two Stores

- Read feedback from two separate stores using CSV files.
- Apply union() to merge both DataFrames vertically.
- Both DataFrames must have the same schema for a valid union.
- Show the combined list as result of customer reviews.



Example: Union – Merge Customer Feedback from Two Stores (Solution)

main.py M

Project > main.py > ...

```python
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("Union Example").getOrCreate()

# Load customer feedback from both stores
store1 = spark.read.option("header", "true").csv("store1_feedback.csv", inferSchema=True)
store2 = spark.read.option("header", "true").csv("store2_feedback.csv", inferSchema=True)

# Combine feedback using union
combined = store1.union(store2)

# Show merged result
combined.show()
```

Example: Union – Merge Customer Feedback from Two Stores (Output)

```
+--------+------+
|customer|rating|
+--------+------+
|   Alice|     5|
|     Bob|     4|
| Charlie|     3|
|   Diana|     5|
+--------+------+

coder@dcfacefbacddcf329981901fdbaacaacdabdabdtwo-0:~/project/workspace/Project$
```

Handling Missing Data in DataFrames

Handling Nulls:

- In real-world datasets, missing data (nulls) is common. Spark provides two main functions to handle this:
  - **dropna()**: Removes rows with any null values.
  - **fillna()**: Replaces null values with a specified value.

Example:

```
# DataFrame with missing values
df = spark.createDataFrame([(1, None), (2, "Sales")], ["id", "department"])
# Replacing nulls with a default value
df_filled = df.fillna("Unknown", subset=["department"])
df_filled.show()
```

Output:

```
+--+------+
| id|department|
+--+------+
|  1|  Unknown|
|  2|   Sales|
+--+------+
```

Introduction to Spark SQL

Overview of Spark SQL:
Spark SQL allows you to query data in DataFrames using SQL syntax, offering the best of both worlds: the efficiency of Spark's Catalyst optimizer and the expressiveness of SQL.

Benefits:

- Easy-to-use SQL syntax for data manipulation.
- Powerful integration with other BI tools.
- Can perform complex ETL tasks.

Accessing Spark SQL:

You can use Spark SQL by registering a DataFrame as a temporary table and then querying it using SQL queries:

```
df.createOrReplaceTempView("employees")
result = spark.sql("SELECT name FROM employees WHERE salary > 50000")
result.show()
```

Running SQL Queries in Spark

Example:

1. Register a DataFrame as a temporary table.
2. Run SQL queries to fetch data.

```
# Register DataFrame as temp view
df.createOrReplaceTempView("employees")

# Run SQL query
result_df = spark.sql("SELECT name FROM employees WHERE salary > 50000")
result_df.show()
```

Common SQL Operations in Spark

Common SQL Operations:

- **Filtering**: SQL allows using WHERE to filter data.
  - **SELECT * FROM employees WHERE department = 'Sales'**
- **Aggregation**: You can use GROUP BY and aggregate functions like AVG, SUM, etc.
  - **SELECT department, AVG(salary) FROM employees GROUP BY department**
- **Joining Tables**: SQL can also perform joins between multiple tables.
  - **SELECT e.name, d.department**
  - **FROM employees e**
  - **JOIN departments d ON e.dept_id = d.id**

Spark SQL as an ETL Tool

ETL (Extract, Transform, Load):
Spark SQL is an excellent tool for ETL tasks, where you load data, apply transformations, and then store the results.

Workflow:
1. **Load Data**: Use Spark SQL to load data from various sources (CSV, Parquet, etc.).
2. **Cleanse and Transform**: Apply transformations like filtering, aggregation, and joining.
3. **Write Data**: Save the processed data to the target system, such as a database or a file system.

Example:

```
# Load data from SQL
df = spark.sql("SELECT * FROM sales_data WHERE sales > 1000")

# Write to parquet
df.write.format("parquet").save("/output/path")
```

Spark SQL Performance Tuning

Optimization Strategies:

- **Partitioning**: Partition your DataFrames by certain columns to distribute the data more evenly across nodes.
- **Caching**: Cache frequently accessed DataFrames to improve performance for iterative operations.
- **Broadcast Joins**: For smaller lookup tables, broadcast them to all nodes to avoid shuffling.

Example of Broadcasting:

```
from pyspark.sql import functions as F

# Broadcast smaller DataFrame for more efficient join
df_large = spark.read.csv("large_file.csv")
df_small = spark.read.csv("small_lookup.csv")

# Perform a broadcast join
joined_df = df_large.join(F.broadcast(df_small), "key_column")
```

Hands-On Exercise - Advanced Operations

Exercise Objective: Write Spark SQL queries to perform the following:
Find the average sales by region:

```
SELECT region, AVG(sales) AS avg_sales FROM sales_data GROUP BY region
```

List departments with employee counts greater than 10:

```
SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY department
HAVING employee_count > 10
```

Sample Solution:

```
    # Find average sales by region
avg_sales_by_region = spark.sql("SELECT region, AVG(sales) AS avg_sales FROM sales_data
GROUP BY region")
avg_sales_by_region.show()
    # List departments with more than 10 employees
departments_with_large_teams = spark.sql("SELECT department, COUNT(*) AS employee_count
FROM employees GROUP BY department HAVING employee_count > 10")
departments_with_large_teams.show()
```

Example: Spark SQL to Filter Top-Paid Employees

- Load employee records from the CSV file into a Spark DataFrame.
- Register the DataFrame as a temporary SQL view named "employees".
- Execute SQL query: filters and selects employees earning more than ₹60,000.

main.py M | employees.csv U

Project > employees.csv

	A	B	C
1	**name**	**departm...**	**salary**
2	Alice	Engineeri...	75000
3	Bob	Marketing	52000
4	Charlie	Engineeri...	87000
5	Diana	HR	48000

Example: Spark SQL to Filter Top-Paid Employees (Solution)

main.py M × employees.csv U

Project > main.py > ...

```python
from pyspark.sql import SparkSession

# Initialize Spark session
spark = SparkSession.builder.appName("Spark SQL Example").getOrCreate()

# Load employee data from CSV
df = spark.read.option("header", "true").csv("employees.csv", inferSchema=True)

# Register DataFrame as temporary SQL view
df.createOrReplaceTempView("employees")

# Run SQL query to filter employees with salary > 60000
result = spark.sql("SELECT name, salary FROM employees WHERE salary > 60000")

# Show result
result.show()
```

Example: Spark SQL to Filter Top-Paid Employees (Output)

```
+-------+------+
|   name|salary|
+-------+------+
|  Alice| 75000|
|Charlie| 87000|
+-------+------+

coder@dcfacefbacddcf329981901fdbaacaacdabdabdtwo-0:~/project/workspace/Project$
```



Thank You